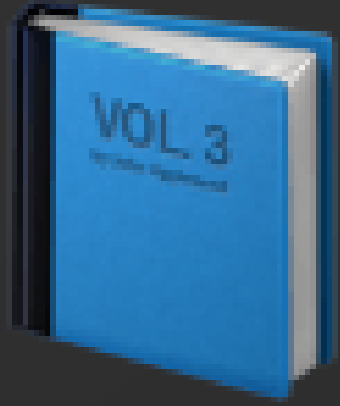




Lesson: Loops in Python

Introduction

In programming, you often need to repeat the same task multiple times.

For example:

- Printing a message 10 times
- Processing items in a list
- Reading multiple lines from a file

Instead of writing the same code over and over, Python provides **loops**, which allow you to repeat a block of code automatically.

👉 Loops are one of the most important concepts in programming because they help you:

- Save time
- Reduce errors
- Write more powerful programs

In this lesson, you will learn how to use loops in Python and when to use each type.



What Is a Loop?

A **loop** is a structure that repeats a block of code.
Instead of writing this:

```
print(Hello)  
print(Hello)  
print(Hello)
```

You can write:

```
for i in range(3):  
    print(Hello)
```



Types of Loops in Python

Python has two main types of loops:

1. **for loops** — used when you know how many times to repeat
2. **while loops** — used when you repeat until a condition is met

The **for** Loop

When to Use It

Use a **for loop** when:

- You know how many times to repeat
- You are working with a list or collection



Example 1 — Basic `for` Loop

```
for i in range(5):  
    print>Hello, World!)
```



Output

```
Hello, World!  
Hello, World!  
Hello, World!  
Hello, World!  
Hello, World!
```

Explanation

- `range(5)` generates numbers from 0 to 4
- The loop runs 5 times
- `i` is a variable that changes each time



Example 2 — Using the Loop Variable

```
for i in range(5):  
    print(Loop number:, i)
```



Output

```
Loop number: 0  
Loop number: 1  
Loop number: 2  
Loop number: 3  
Loop number: 4
```




Example 3 — Looping Through a List

```
names = [Alice, Bob, Charlie]
for name in names:
    print>Hello,, name)
```



Output

```
Hello, Alice
Hello, Bob
Hello, Charlie
```

Common Use Cases for `for` Loops

- Repeating a task a fixed number of times
- Processing items in a list
- Iterating through strings or files

The **while** Loop

When to Use It

Use a **while loop** when:

- You don't know how many times the loop will run
- You want to repeat until a condition changes



Example 1 — Basic `while` Loop

```
count = 0
while count < 5:
    print(Count is:, count)
    count = count + 1
```



Output

```
Count is: 0
Count is: 1
Count is: 2
Count is: 3
Count is: 4
```

Explanation

- The loop runs while `count < 5` is true
- Each time through the loop, `count` increases
- When `count` reaches 5, the loop stops

⚠ Important Concept: Infinite Loops

If the condition never becomes false, the loop will run forever.

Example of an infinite loop:



```
while True:  
    print(This will run forever)
```

👉 Always make sure your loop has a **way to stop**.



Example 2 — User Input Loop



```
name =  
while name == :  
    name = input(Enter your name: )  
print(Hello,, name)
```



What This Does

- Keeps asking until the user enters a name
- Stops when a valid input is provided

Common Use Cases for `while` Loops

- Waiting for user input
- Running until a condition is met
- Menu-driven programs



Controlling Loops

Python provides special statements to control loop behavior.

◆ `break` — Exit the Loop Early

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

✅ Output

```
0  
1  
2  
3  
4
```

👉 The loop stops when `i == 5`

◆ `continue` — Skip an Iteration

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

✓ Output

```
0  
1  
3  
4
```

👉 Skips the value `2`



Comparing `for` vs `while`

Feature	<code>for</code> Loop	<code>while</code> Loop
Best Use Case	Known number of repetitions	Unknown repetitions
Common Use	Lists, ranges	Conditions, user input
Risk	Low	Can become infinite



Building a Program with Loops

Let's combine what you've learned.



Example — Greeting Multiple Users

```
names = []  
for i in range(3):  
    name = input(Enter a name: )  
    names.append(name)  
print(\nGreetings:)  
for name in names:  
    print>Hello,, name)
```

Example Output



```
Enter a name: Alice
Enter a name: Bob
Enter a name: Charlie
Greetings:
Hello, Alice
Hello, Bob
Hello, Charlie
```


What This Demonstrates

- Collecting input in a loop
- Storing values in a list
- Looping through the list

 This is starting to look like a **real program**

Summary

In this lesson, you learned:

- What loops are and why they are important
- How to use a `for` loop to repeat a fixed number of times
- How to use a `while` loop to repeat based on a condition
- How to control loops using `break` and `continue`
- Common use cases for each type of loop

Final Thought

“

Loops are where programming becomes powerful.

Instead of writing repetitive code, you can:

- Automate tasks
- Process large amounts of data
- Build interactive programs

👉 Mastering loops is a major step toward becoming a confident Python developer.